

Association Rules

Part 2

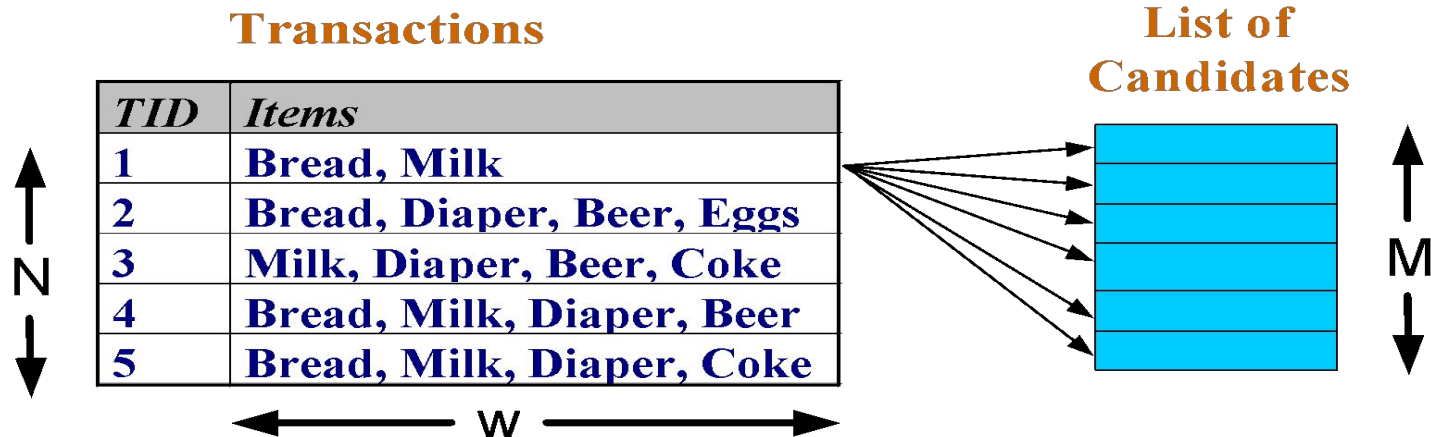
Mohammed Brahimi & Sami Belkacem

Outline

1. Frequent Itemsets and Association Rules
- 2. Apriori Algorithm**
3. FP-Growth Algorithm
4. Evaluation of Association Patterns

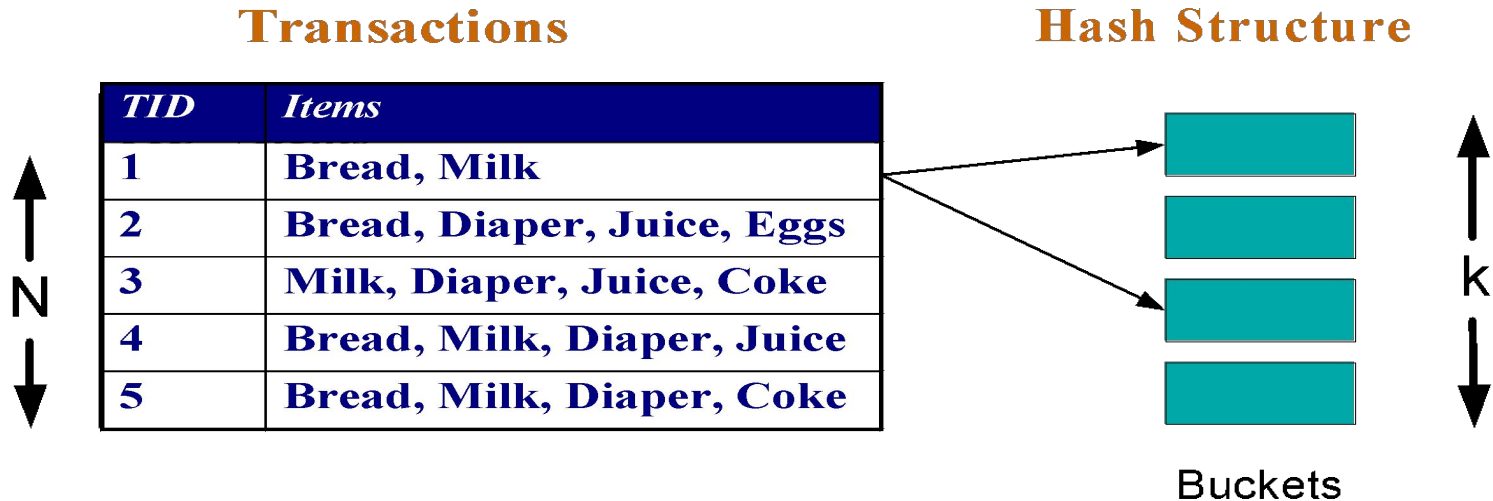
Support Counting of Candidate Itemsets

- Scan the all transactions to determine the support of each candidate itemset.
 - Match every candidate itemset against every transaction.
 - **N**: # of transactions, **M**: # of candidate itemsets, **w**: maximum transaction width
- ⇒ Complexity $\sim O(NMw) \Rightarrow$ Expensive since $M = 2^d$!!!



Support Counting of Candidate Itemsets

- To reduce number of comparisons, store the candidate itemsets in a hash structure.
- Instead of matching each transaction against every candidate, match it against candidates stored in the hashed buckets.



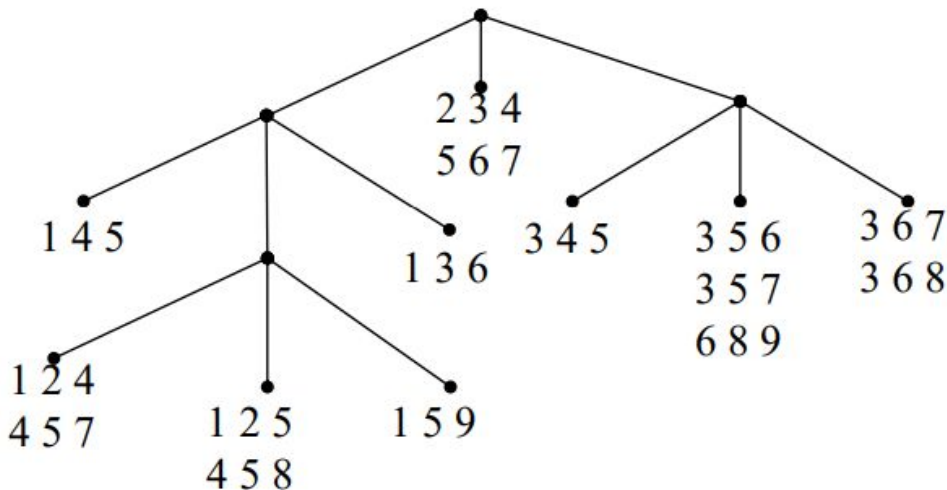
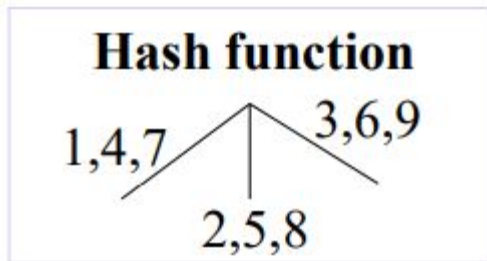
High-level Apriori loop

For each level k :

1. **Generate candidate itemsets C_k**
2. **Build a hash tree on C_k**
3. **Scan the database**
4. **For each transaction t :**
 - Enumerate all k -subsets of t
 - Use the hash tree to check whether each subset is in C_k
 - Increment the support if found
5. **Prune $\rightarrow$ get F_k**

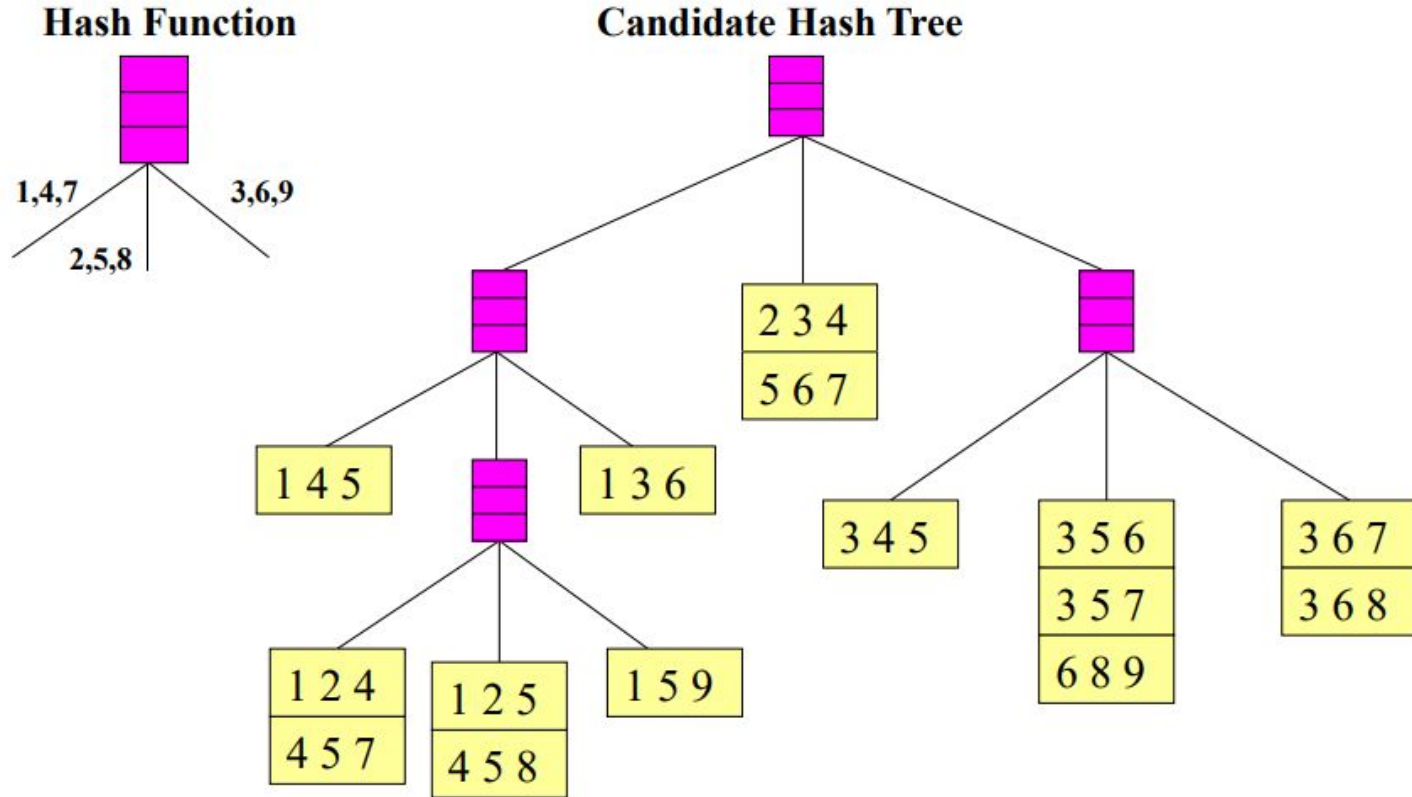
Support Counting Using Hash Tree

- 15 candidate itemsets of length 3:
 - {1 4 5}, {1 2 4}, {4 5 7}, {1 2 5}, {4 5 8},
 - {1 5 9}, {1 3 6}, {2 3 4}, {5 6 7}, {3 4 5},
 - {3 5 6}, {3 5 7}, {6 8 9}, {3 6 7}, {3 6 8}.
- We need Hash function to locate the bucket
 - HashFunc: mod 3
 - Bucket size = 3 → Any node containing more than three candidates must be split.



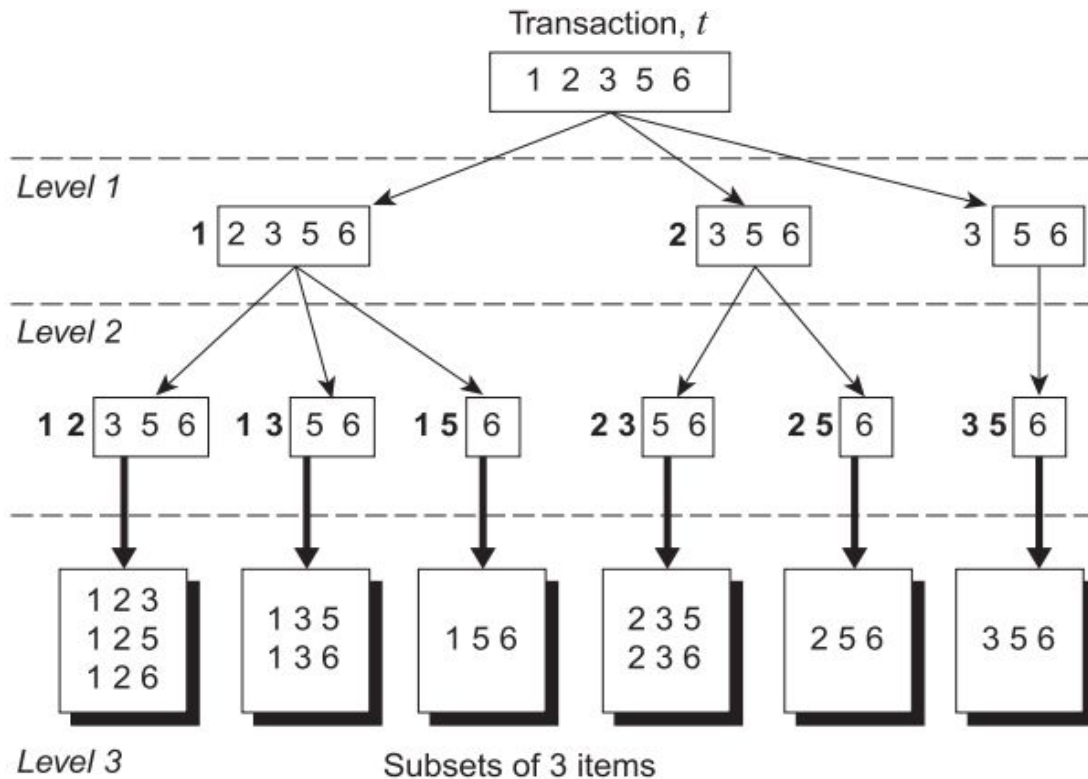
Support Counting Using Hash Tree

Generate Candidate Hash Tree



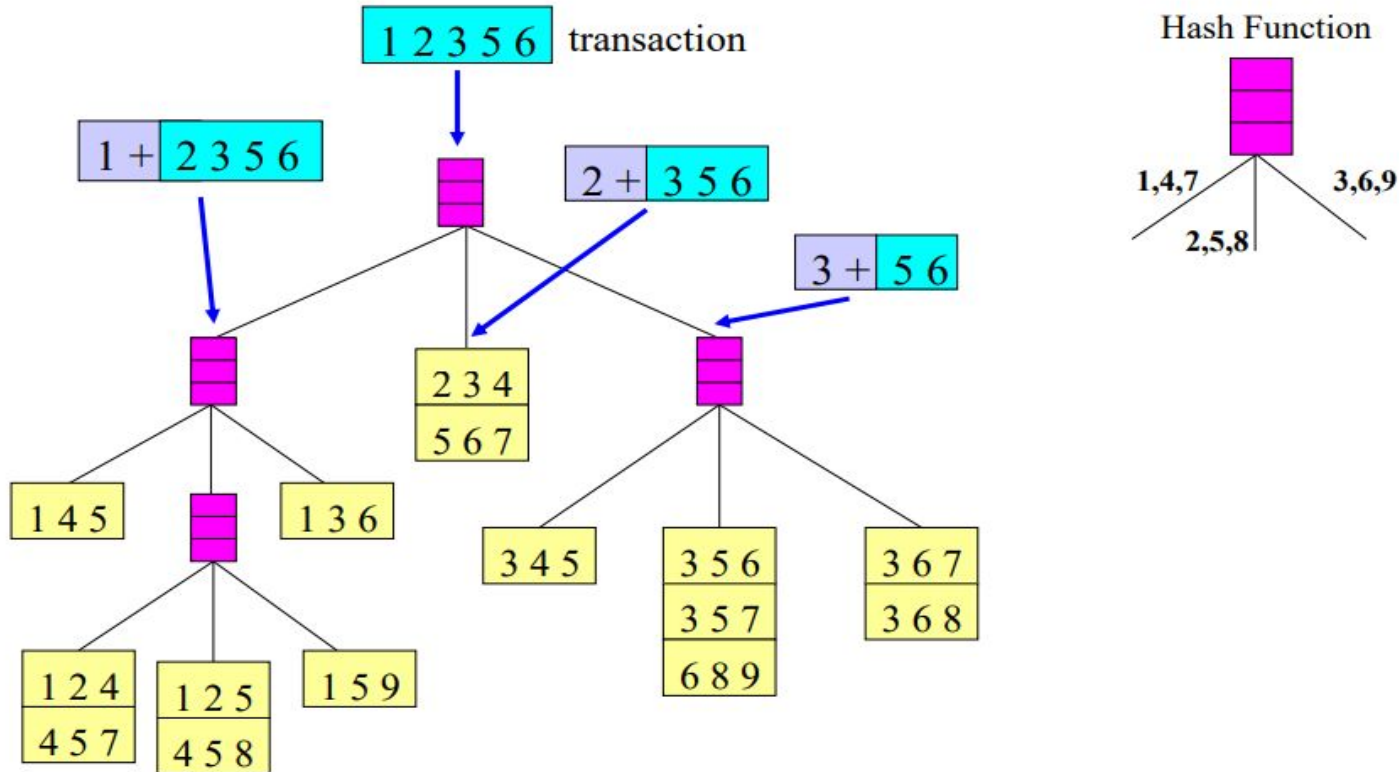
Support Counting Using Hash Tree

- Enumerating subsets of three items from a transaction t



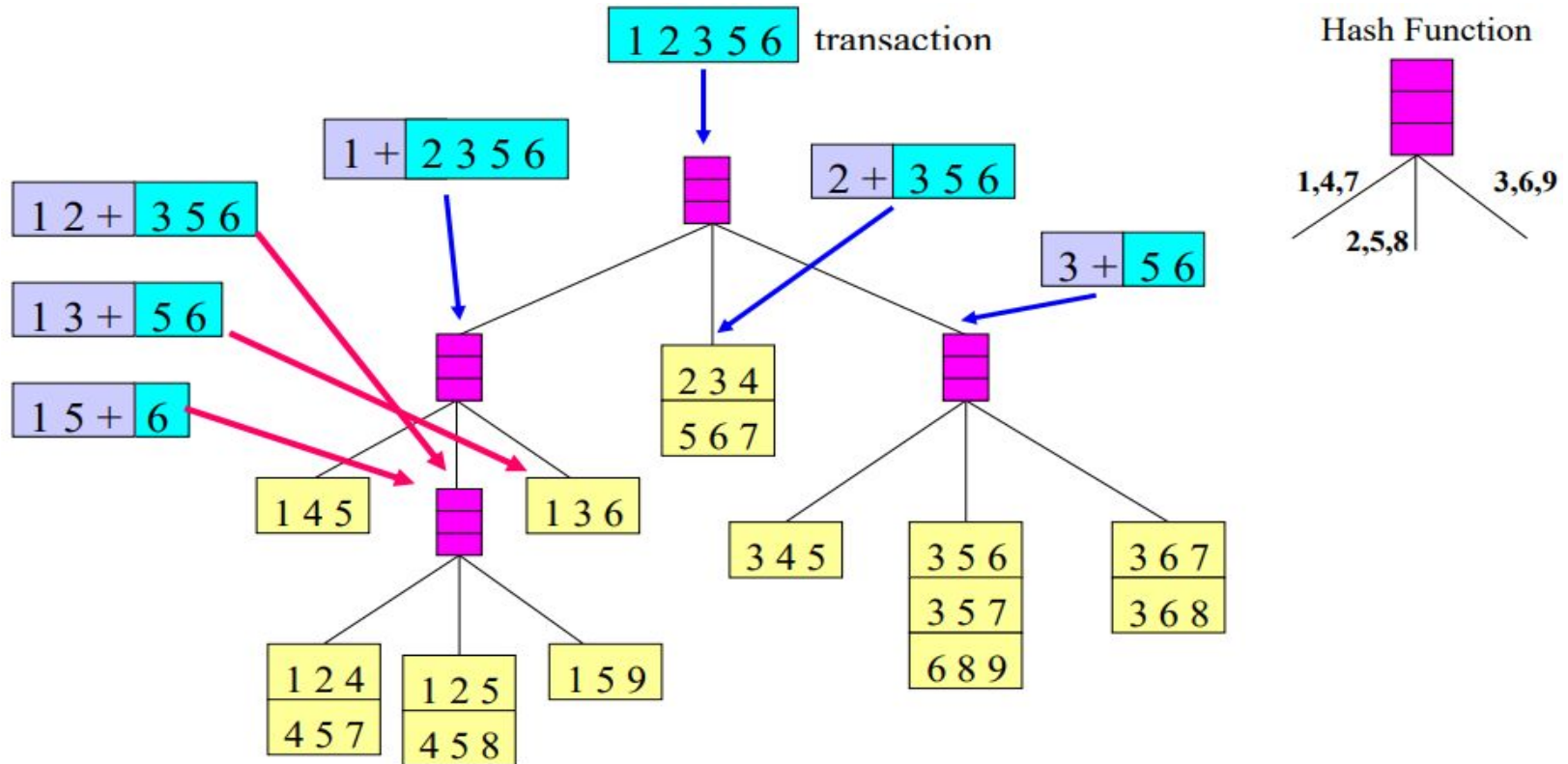
Support Counting Using Hash Tree

Traverse Candidate Hash Tree to Update Support Counts



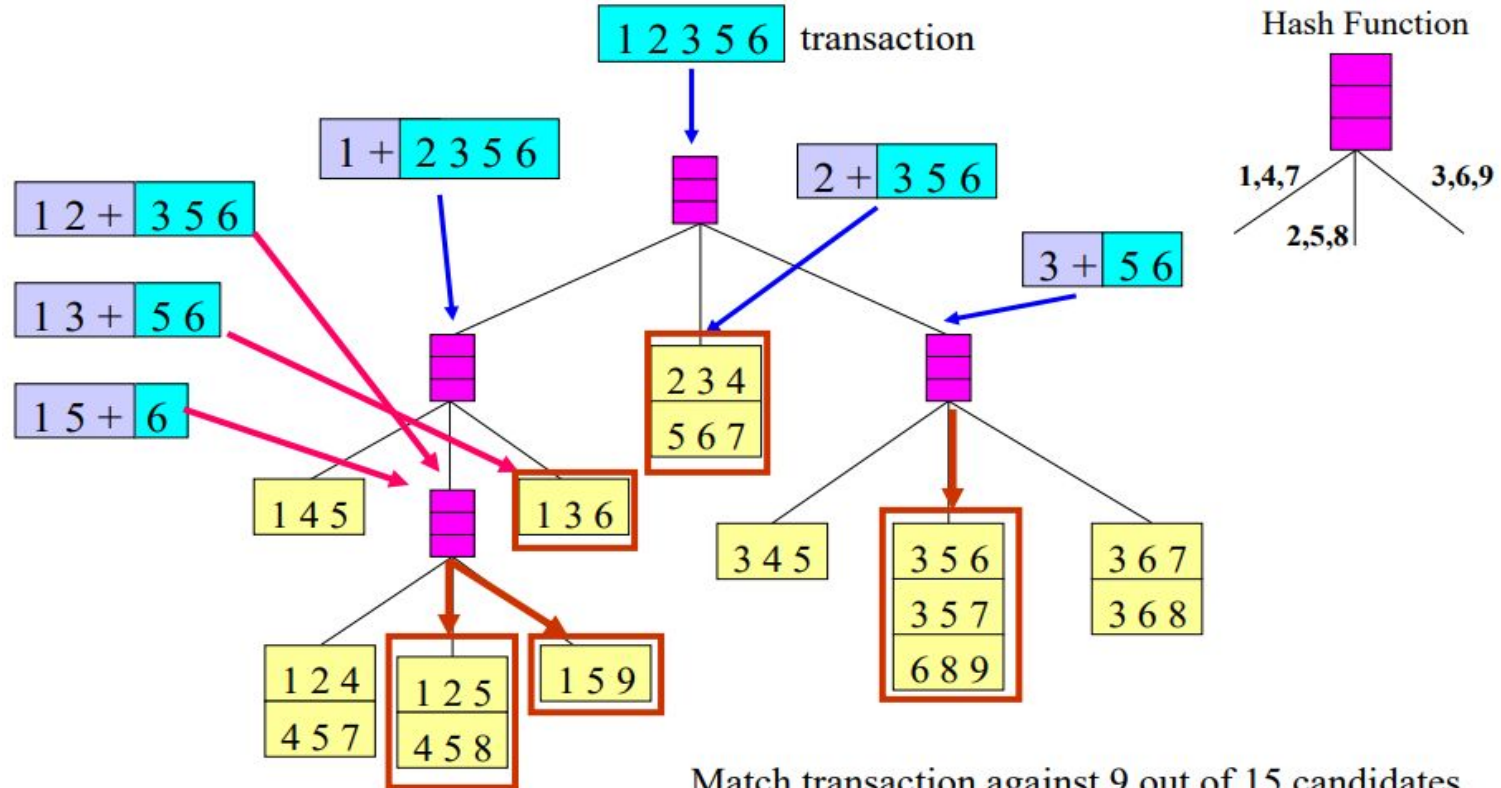
Support Counting Using Hash Tree

Traverse Candidate Hash Tree to Update Support Counts



Support Counting Using Hash Tree

Traverse Candidate Hash Tree to Update Support Counts



Rule Generation

- Given a frequent itemset L , find all non-empty subsets $f \subset L$ such that:
 $f \rightarrow L - f$ satisfies the minimum confidence requirement
- If $\{A,B,C,D\}$ is a frequent itemset, candidate rules:
 $ABC \rightarrow D, \quad ABD \rightarrow C, \quad ACD \rightarrow B, \quad BCD \rightarrow A,$
 $A \rightarrow BCD, \quad B \rightarrow ACD, \quad C \rightarrow ABD, \quad D \rightarrow ABC$
 $AB \rightarrow CD, \quad AC \rightarrow BD, \quad AD \rightarrow BC, \quad BC \rightarrow AD,$
 $BD \rightarrow AC, \quad CD \rightarrow AB,$
- If $|L| = k$, then there are $2^k - 2$ candidate association rules.
 - Ignoring $L \rightarrow \emptyset$ and $\emptyset \rightarrow L$

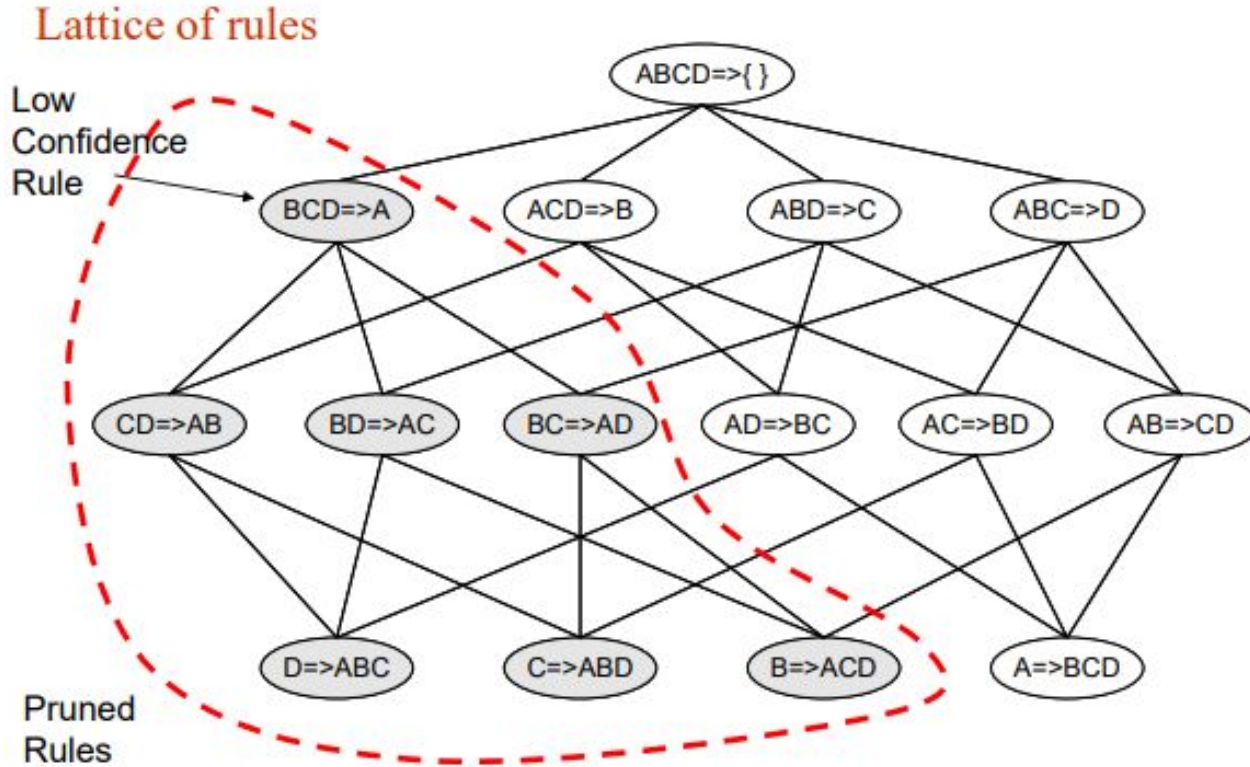
Apriori Algorithm: Rule Generation

- How to efficiently generate rules from frequent itemsets?
- In general, confidence does not have an anti-monotone property:

$c(ABC \Rightarrow D)$ can be larger or smaller than $c(AB \Rightarrow D)$

- But confidence of rules generated from the same itemset has an anti-monotone property
 - E.g., Suppose $\{A,B,C,D\}$ is a frequent 4-itemset: **$c(ABC \Rightarrow D) \geq c(AB \Rightarrow CD) \geq c(A \Rightarrow BCD)$**
 - Confidence is anti-monotone with respect to the number of items on the RHS of the rule.

Apriori Algorithm: Rule Generation



Outline

1. Frequent Itemsets and Association Rules
2. Apriori Algorithm
3. **FP-Growth Algorithm**
4. Evaluation of Association Patterns

FP-Growth (Frequent Pattern Growth) Algorithm

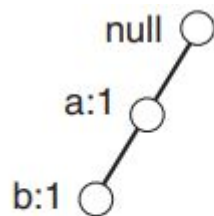
- **Alternative to Apriori:**
 - FP-Growth does not follow the generate-and-test paradigm used by Apriori.
 - FP-Growth still generates the same frequent itemsets as Apriori.
 - The rule generation step is identical for both algorithms.
- **Compressed Database Representation.**
 - The algorithm uses an FP-tree to compress the transaction database
 - Improve computational efficiency while capturing relationships between items.
- **FP-Tree: A Compact Data Structure**
 - The FP-tree encodes the dataset efficiently
 - Direct extraction of frequent itemsets without the need for candidate generation.

FP-Tree Construction

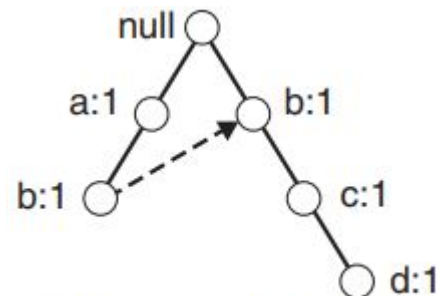
Transaction
Data Set

TID	Items
1	{a,b}
2	{b,c,d}
3	{a,c,d,e}
4	{a,d,e}
5	{a,b,c}
6	{a,b,c,d}
7	{a}
8	{a,b,c}
9	{a,b,d}
10	{b,c,e}

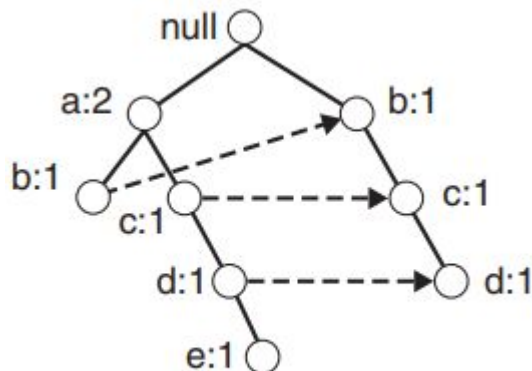
min_sup = 2



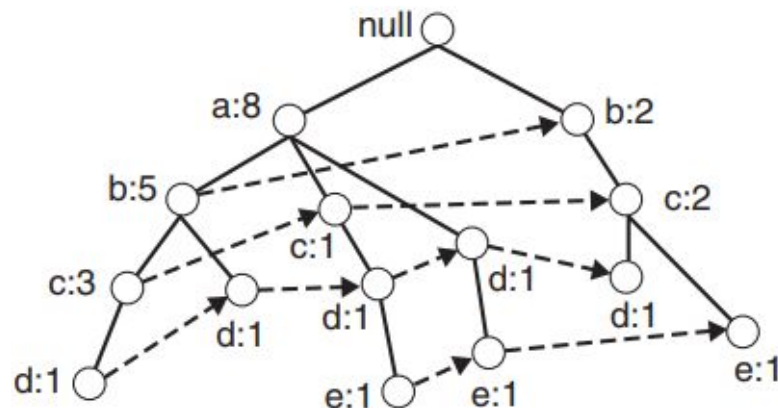
(i) After reading TID=1



(ii) After reading TID=2



(iii) After reading TID=3



(iv) After reading TID=10

FP-Tree Construction

Step 1: Initial Scan to Determine Support Counts

- Traverse the dataset once to calculate the **support count** for each item.

Step 2: Filter and Sort Items

- Discard items that do not meet the minimum support threshold.
- Sort the remaining (frequent) items in **descending order of support counts**.

Step 3: Build the FP-Tree

1. **Second Scan:** Go through the dataset again to construct the FP-tree.
2. **Node Creation:** For each transaction, add nodes for its frequent items in sorted order.
3. **Path Formation:** Link nodes to represent each transaction.
4. **Frequency Update:** Increment counts for nodes when transactions share prefixes.
5. **Path Separation:** Start new paths for transactions that don't share a prefix.

Repeat steps **3.1 to 3.5** for every transaction to build the complete FP-tree.

FP-Tree Construction

Step 1: Initial Scan to Determine Support Counts

- Traverse the dataset once to calculate the **support count** for each item.

Step 2: Filter and Sort Items

- Discard items with support count less than min support.
- Sort the remaining items in descending order of support count.

Step 3: Build the FP-Tree

1. Second Scan: Count the frequency of each item in the dataset.
2. Node Creation: Create a node for each item in the dataset.
3. Path Formation: Form paths for each transaction.

When many transactions share the same path in the FP-tree, it shows that those items commonly occur together, making the corresponding itemsets frequent.

4. **Frequency Update:** Increment counts for nodes when transactions share prefixes.
5. **Path Separation:** Start new paths for transactions that don't share a prefix.

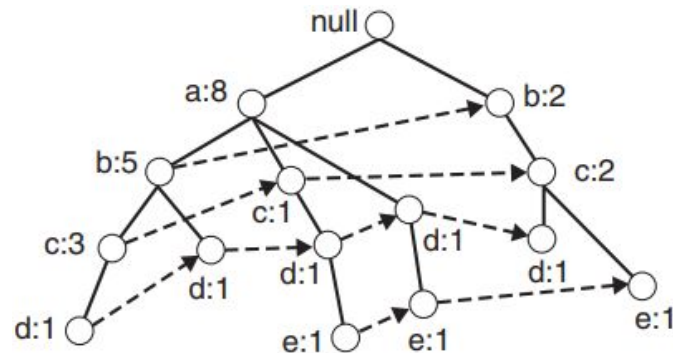
Repeat steps 3.1 to 3.5 for every transaction to build the complete FP-tree.

FP-Tree Construction and Item Ordering

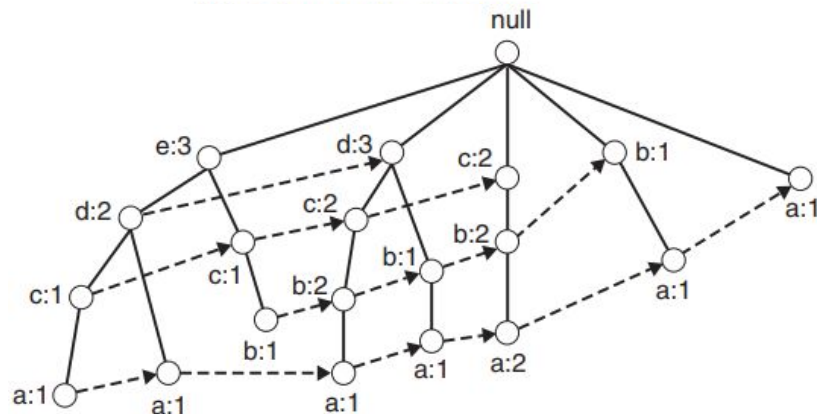
Transaction
Data Set

TID	Items
1	{a,b}
2	{b,c,d}
3	{a,c,d,e}
4	{a,d,e}
5	{a,b,c}
6	{a,b,c,d}
7	{a}
8	{a,b,c}
9	{a,b,d}
10	{b,c,e}

**Order 1
starts by a**



**Order 2
starts by e**



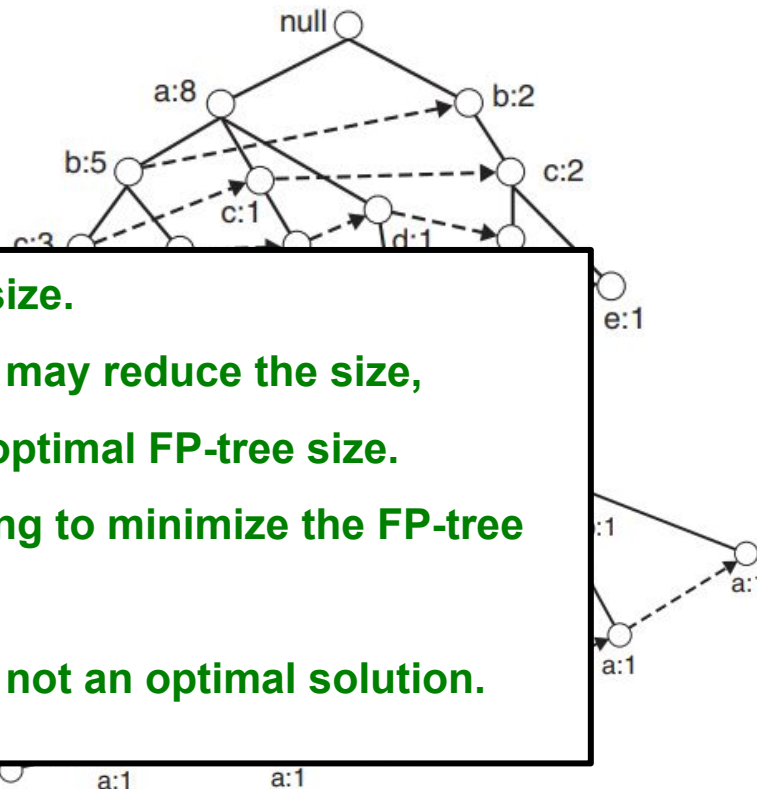
FP-Tree construction and the order of items

Transaction
Data Set

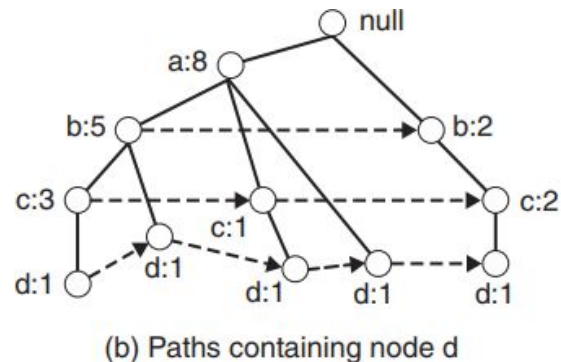
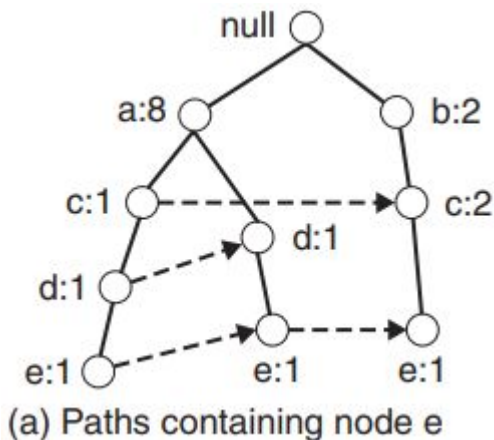
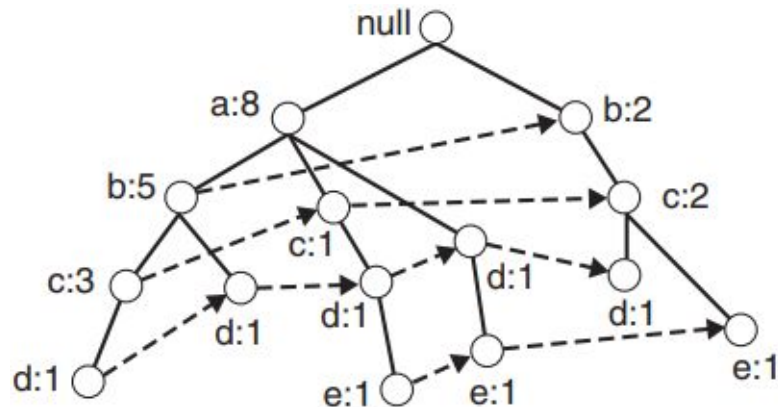
TID	Items
1	{a,b}
2	{b,c,d}
3	{a,c,d,e}
4	{a,d,e}
5	{a,b,c}
6	{a,b,c,d}
7	{a}
8	{a,b,c}
9	{a,b,d}
10	{b,c,e}

Order 1

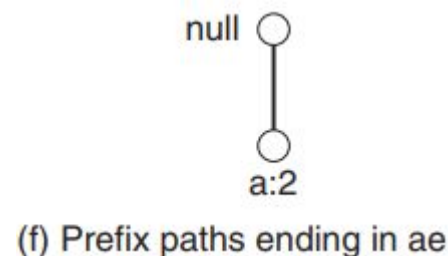
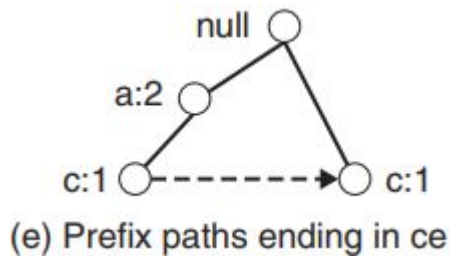
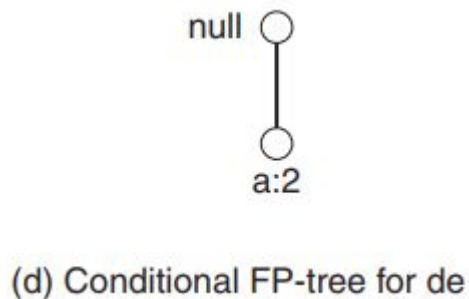
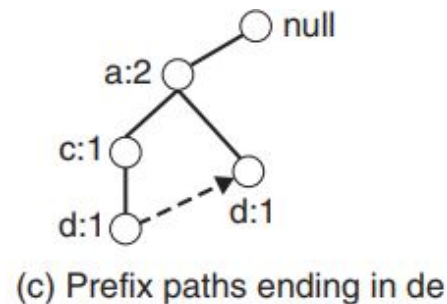
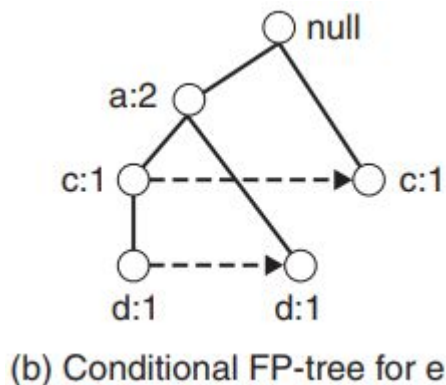
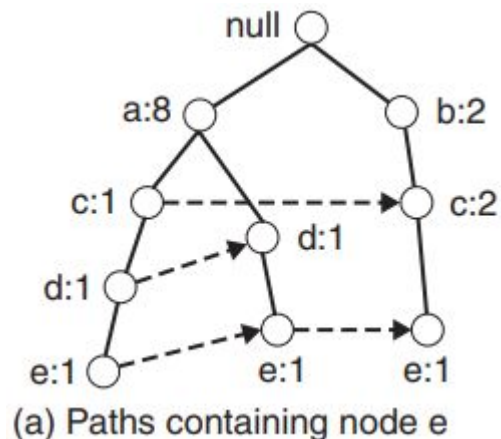
- Item order impacts FP-tree size.
- Starting with frequent items may reduce the size, but it doesn't guarantee an optimal FP-tree size.
- Finding the best item ordering to minimize the FP-tree size is an NP-hard problem.
- FP-Growth uses a heuristic, not an optimal solution.



How to Count the Support of a Pattern



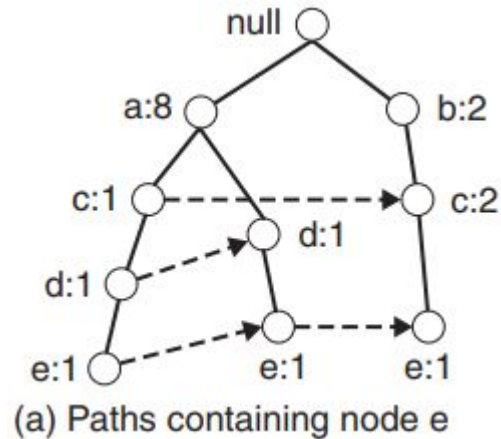
Frequent Itemset Generation in FP-Growth Algorithm



Conditional FP-Tree ending by “e”

Transaction
Data Set

TID	Items
1	{a,b}
2	{b,c,d}
3	{a,c,d,e}
4	{a,d,e}
5	{a,b,c}
6	{a,b,c,d}
7	{a}
8	{a,b,c}
9	{a,b,d}
10	{b,c,e}

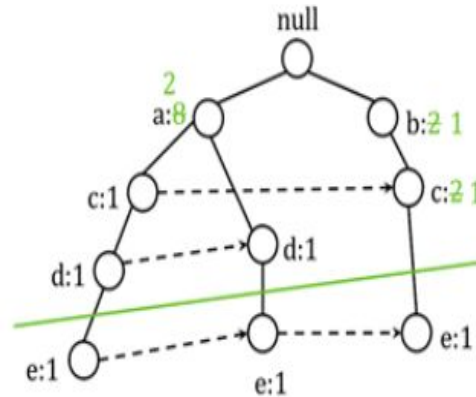


- Keep only paths ending by “e”

Conditional FP-Tree ending by “e”

Transaction
Data Set

TID	Items
1	{a,b}
2	{b,c,d}
3	{a,c,d,e}
4	{a,d,e}
5	{a,b,c}
6	{a,b,c,d}
7	{a}
8	{a,b,c}
9	{a,b,d}
10	{b,c,e}



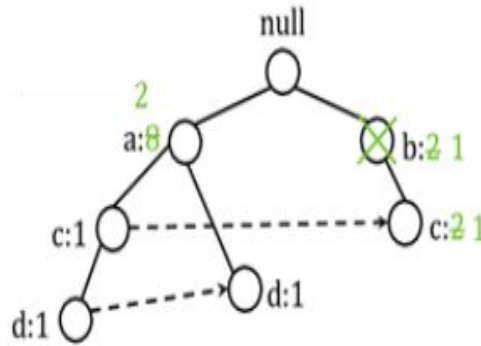
(a) Paths containing node e

- Remove the nodes containing “e”.
- This information is no longer needed.

Conditional FP-Tree ending by “e”

Transaction
Data Set

TID	Items
1	{a,b}
2	{b,c,d}
3	{a,c,d,e}
4	{a,d,e}
5	{a,b,c}
6	{a,b,c,d}
7	{a}
8	{a,b,c}
9	{a,b,d}
10	{b,c,e}



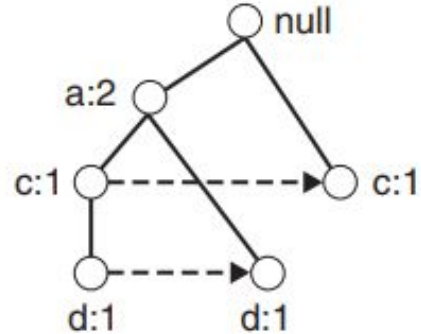
(a) Paths containing node e

- Update the frequencies
- Remove infrequent patterns

Conditional FP-Tree ending by “e”

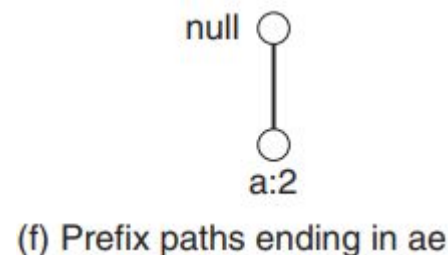
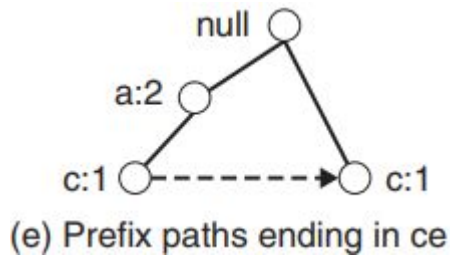
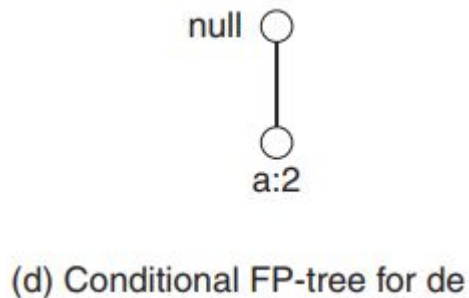
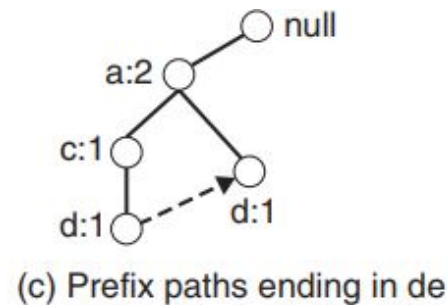
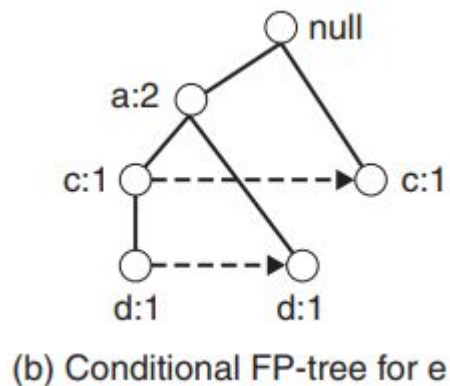
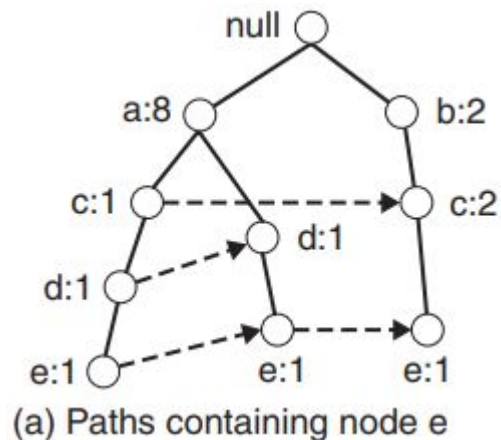
Transaction
Data Set

TID	Items
1	{a,b}
2	{b,c,d}
3	{a,c,d,e}
4	{a,d,e}
5	{a,b,c}
6	{a,b,c,d}
7	{a}
8	{a,b,c}
9	{a,b,d}
10	{b,c,e}

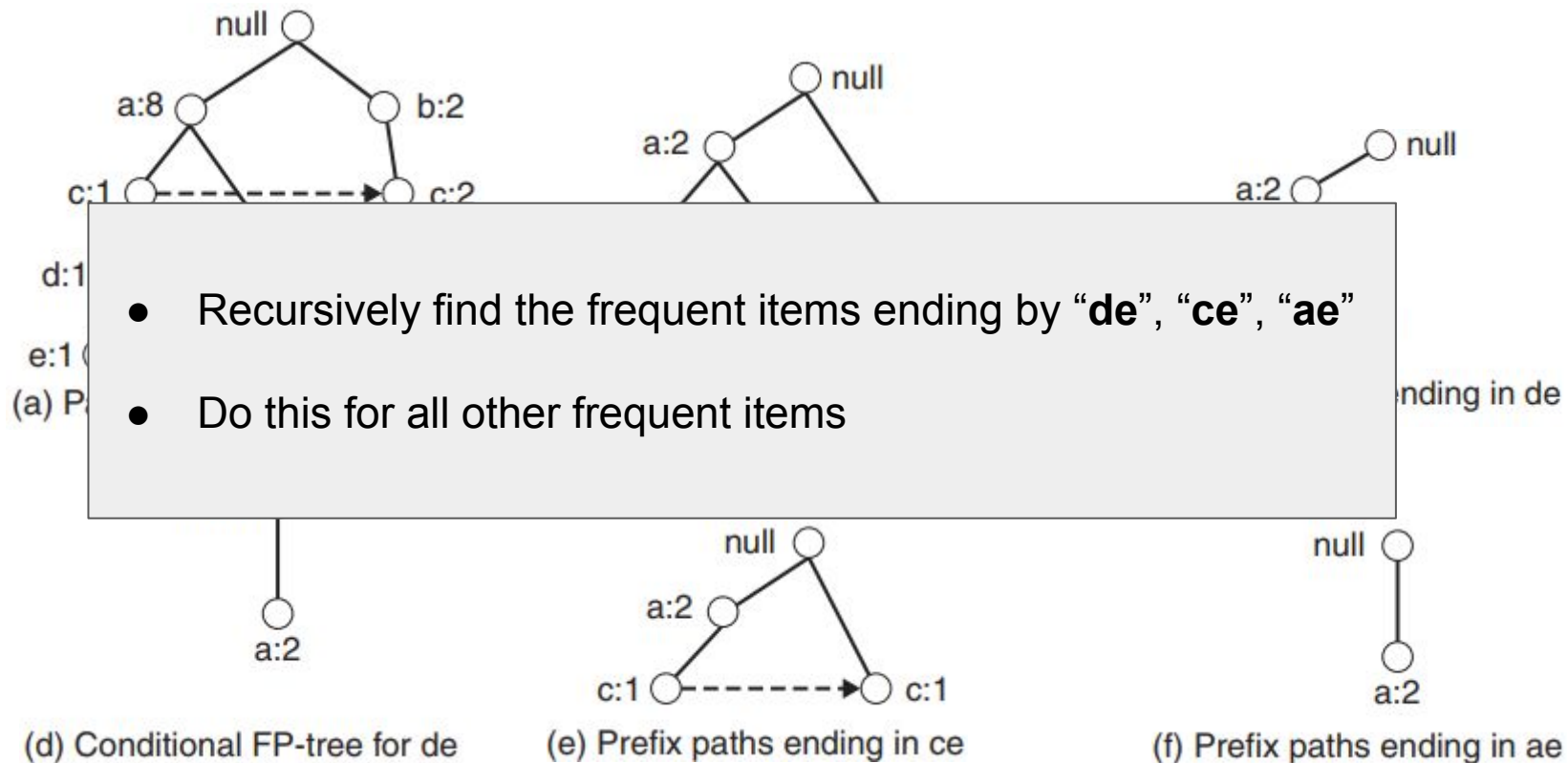


(b) Conditional FP-tree for e

Frequent Itemset Generation in FP-Growth Algorithm



Frequent Itemset Generation in FP-Growth Algorithm



FP-Growth: Example

min support = 3

Transaction ID	Items
T1	E,K,M,N,O,Y
T2	D,E,K,N,O,Y
T3	A,E,K,M
T4	C,K,M,U,Y
T5	C,E,I,K,O



Item	Frequency
K	5
E	4
M	3
O	3
Y	3
C	2
N	2
A	1
D	1
I	1
U	1

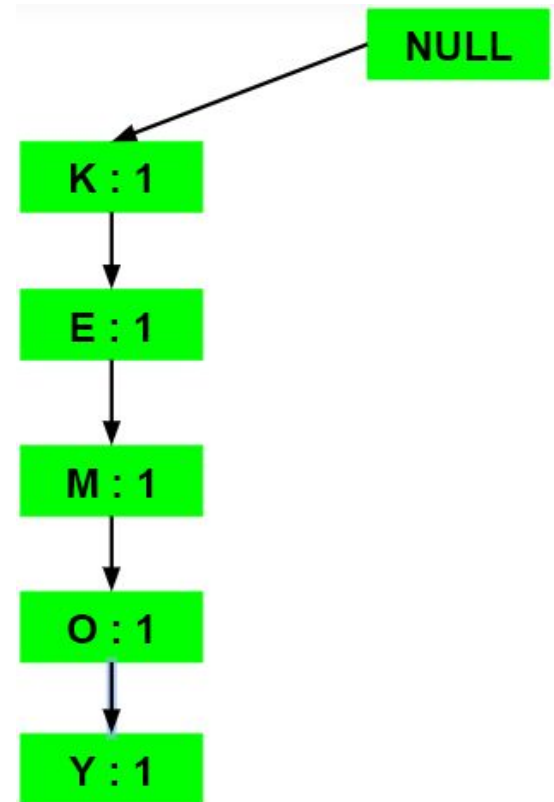
FP-Growth: Example

Item	Frequency
K	5
E	4
M	3
O	3
Y	3

Transaction ID	Items	Ordered-Item set
T1	E,K,M,N,O,Y	K,E,M,O,Y
T2	D,E,K,N,O,Y	K,E,O,Y
T3	A,E,K,M	K,E,M
T4	C,K,M,U,Y	K,M,Y
T5	C,E,I,K,O	K,E,O

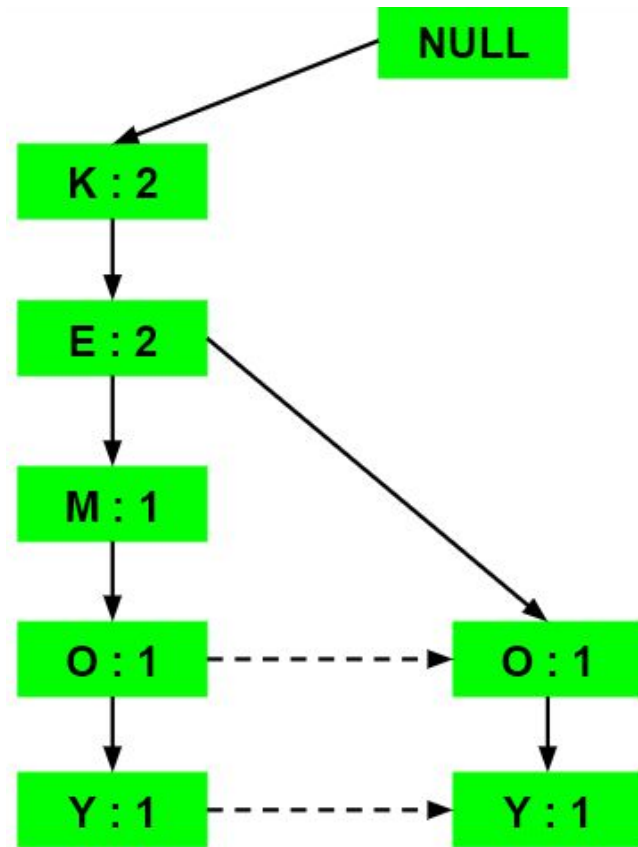
FP-Growth: Example

Transaction ID	Items	Ordered-Item set
T1	E,K,M,N,O,Y	K,E,M,O,Y
T2	D,E,K,N,O,Y	K,E,O,Y
T3	A,E,K,M	K,E,M
T4	C,K,M,U,Y	K,M,Y
T5	C,E,I,K,O	K,E,O



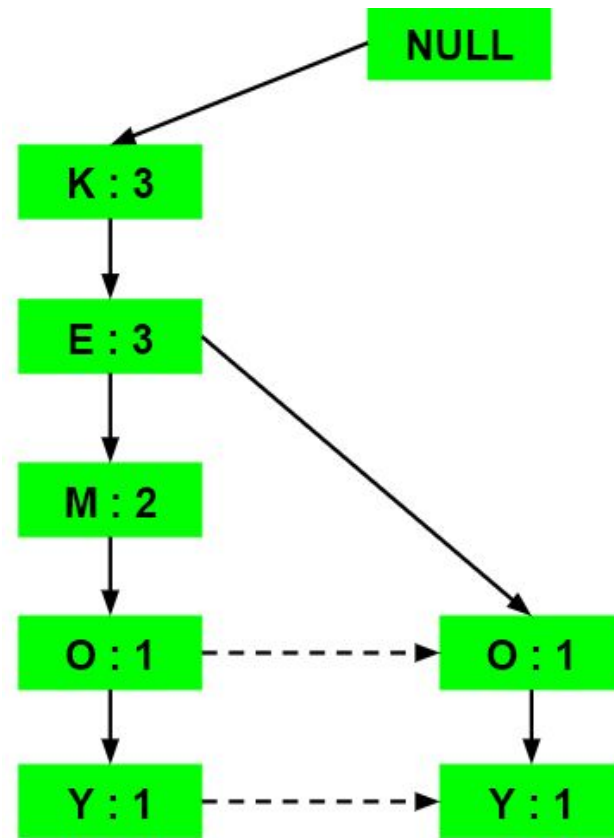
FP-Growth: Example

Transaction ID	Items	Ordered-Item set
T1	E,K,M,N,O,Y	K,E,M,O,Y
T2	D,E,K,N,O,Y	K,E,O,Y
T3	A,E,K,M	K,E,M
T4	C,K,M,U,Y	K,M,Y
T5	C,E,I,K,O	K,E,O



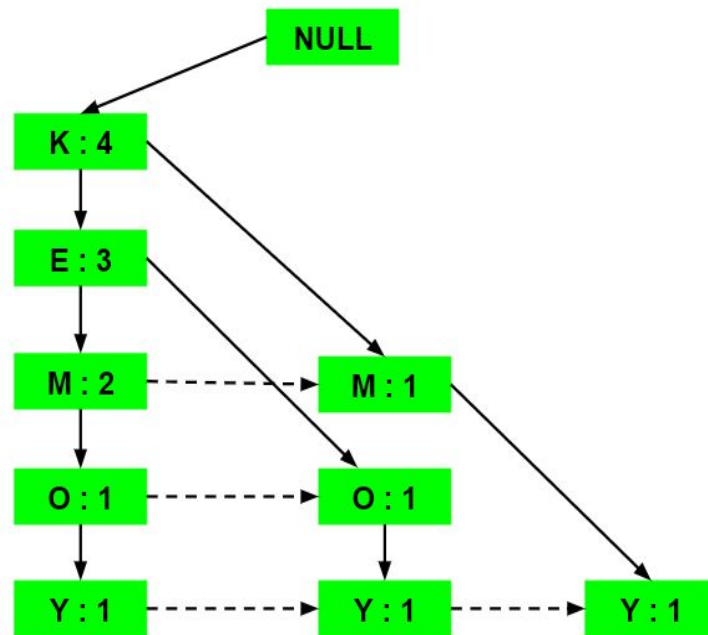
FP-Growth: Example

Transaction ID	Items	Ordered-Item set
T1	E,K,M,N,O,Y	K,E,M,O,Y
T2	D,E,K,N,O,Y	K,E,O,Y
T3	A,E,K,M	K,E,M
T4	C,K,M,U,Y	K,M,Y
T5	C,E,I,K,O	K,E,O



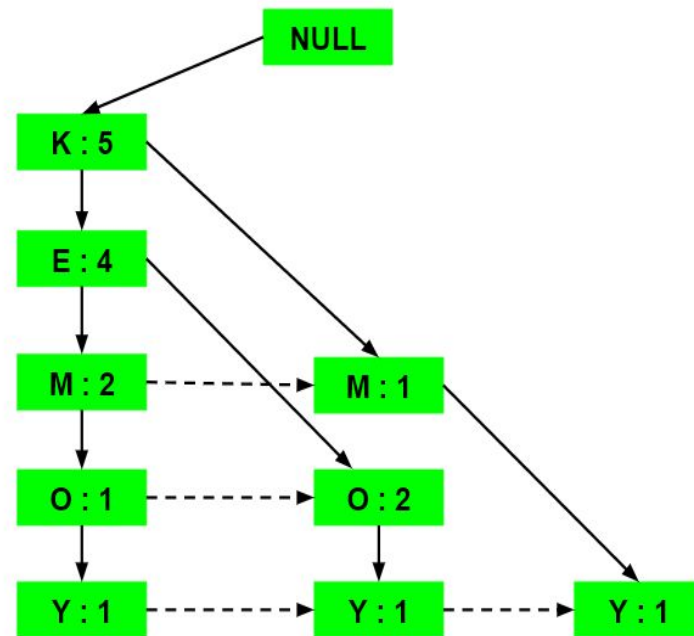
FP-Growth: Example

Transaction ID	Items	Ordered-Item set
T1	E,K,M,N,O,Y	K,E,M,O,Y
T2	D,E,K,N,O,Y	K,E,O,Y
T3	A,E,K,M	K,E,M
T4	C,K,M,U,Y	K,M,Y
T5	C,E,I,K,O	K,E,O



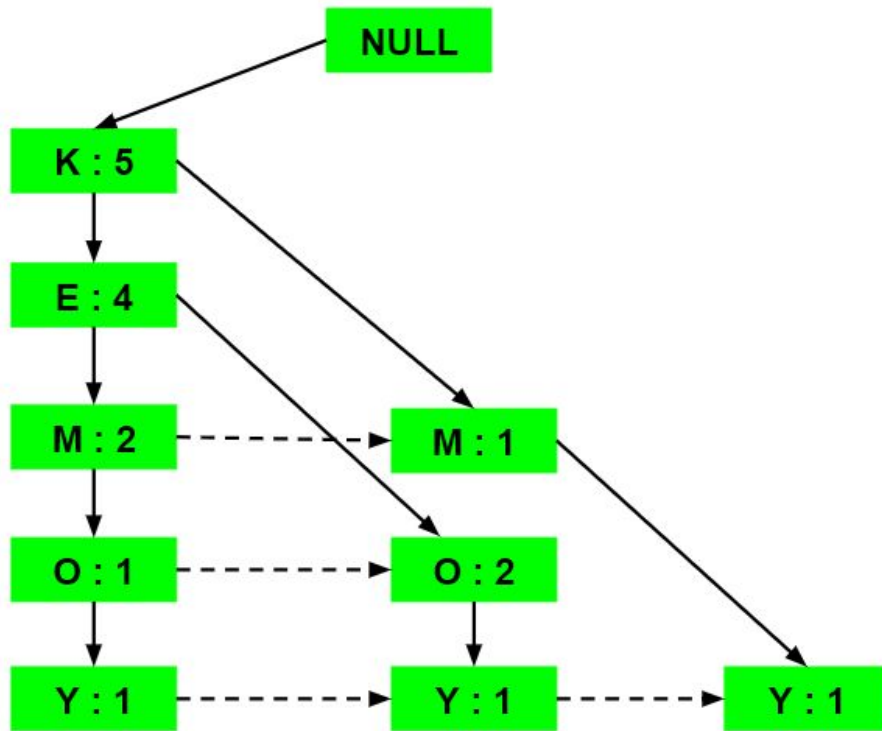
FP-Growth: Example

Transaction ID	Items	Ordered-Item set
T1	E,K,M,N,O,Y	K,E,M,O,Y
T2	D,E,K,N,O,Y	K,E,O,Y
T3	A,E,K,M	K,E,M
T4	C,K,M,U,Y	K,M,Y
T5	C,E,I,K,O	K,E,O



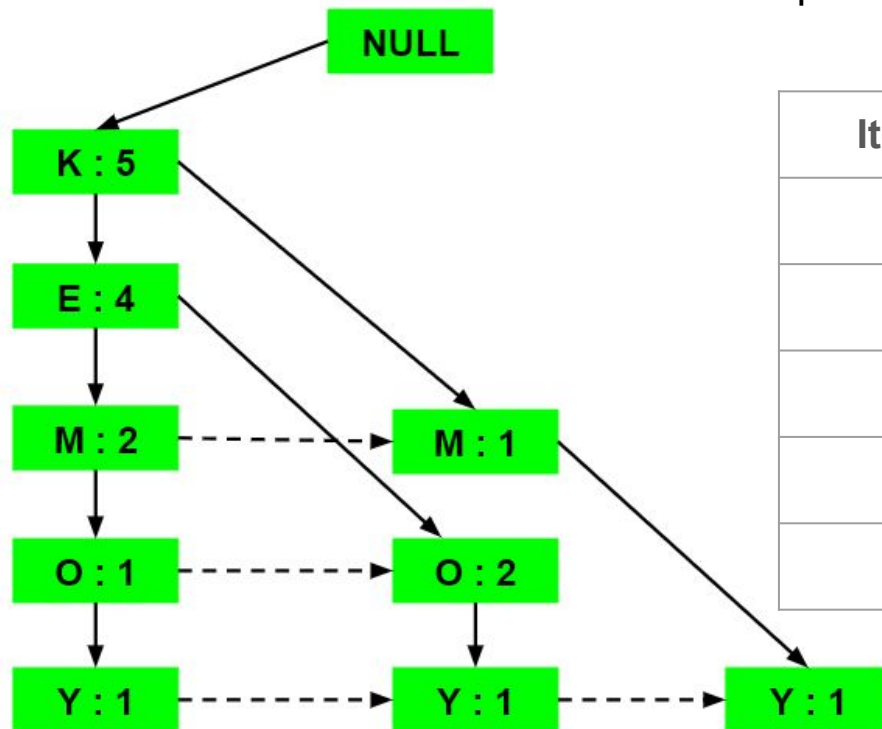
FP-Growth: Example

Transaction ID	Ordered-Item set
T1	K,E,M,O,Y
T2	K,E,O,Y
T3	K,E,M
T4	K,M,Y
T5	K,E,O



FP-Growth: Example

For each frequent item, build its conditional pattern base (from the FP-tree)



Item	Conditional Pattern Base
Y	{{K, E, M, O: 1}, {K, E, O: 1}, {K, M: 1}}
O	{{K, E, M: 1}, {K, E: 2}}
M	{{K, E: 2}, {K: 1}}
E	{K: 4}
K	

FP-Growth: Example

Conditional frequent patterns are obtained by applying minimum support to the conditional pattern base.

Item	Conditional Pattern Base	Conditional Frequent Pattern
Y	{{K, E, M, O: 1}, {K, E, O: 1}, {K, M: 1}}	{K:3}
O	{{K, E, M: 1}, {K, E: 2}}	{K,E:3}
M	{{K, E: 2}, {K: 1}}	{K:3}
E	{K: 4}	{K:4}
K		

FP-Growth: Example

Frequent patterns are generated by combining each item with its conditional frequent patterns.

Item	Conditional Frequent Pattern	Frequent Pattern rules
Y	{K:3}	{K,Y}:3
O	{K,E:3}	{K,O}:3 {E,O}:3 {E,K,O}:3
M	{K:3}	{K,M}:3
E	{K:4}	{E,K}:4
K		

Outline

1. Frequent Itemsets and Association Rules
2. Apriori Algorithm
3. FP-Growth Algorithm
- 4. Evaluation of Association Patterns**

Pattern Evaluation

- Association rule algorithms can produce large number of rules.
- **Interestingness measures** can be used to prune/rank the patterns.
- The purpose is to focus on the most useful patterns.
- In the original formulation, support and confidence are the only measures used.
- In this section, we will introduce other rule evaluation measures.

Computing Interestingness Measure

- Given $X \rightarrow Y$ or $\{X, Y\}$, information needed to compute interestingness can be obtained from a contingency table.

	Y	$\overline{Y}$	
X	f_{11}	f_{10}	f_{1+}
$\overline{X}$	f_{01}	f_{00}	f_{0+}
	f_{+1}	f_{+0}	N

f_{11} : support of X and Y
 f_{10} : support of X and $\overline{Y}$
 f_{01} : support of $\overline{X}$ and Y
 f_{00} : support of $\overline{X}$ and $\overline{Y}$

Contingency table

Used to define various measures:
support, confidence, lift ...

Drawback of Confidence

	Coffee	<u>Coffee</u>	
Tea	150	50	200
<u>Tea</u>	650	150	800
	800	200	1000

Association Rule: Tea $\rightarrow$ Coffee

- Confidence = $P(\text{Coffee}|\text{Tea}) = 150/200 = 0.75$ (seems interesting, but ...)
- $P(\text{Coffee}) = 0.8$ (drinking tea reduces the probability that the person drinks coffee!)
- $P(\text{Coffee}|\overline{\text{Tea}}) = 650/800 = 0.8125$ (not drinking tea increases the probability that the person drinks coffee!)

Drawback of Confidence

Customers	Tea	Honey	...
C1	0	1	...
C2	1	0	...
C3	1	1	...
C4	1	0	...
...			

	<i>Honey</i>	$\overline{Honey}$	
<i>Tea</i>	100	100	200
$\overline{Tea}$	20	780	800
	120	880	1000

Association Rule: Tea $\rightarrow$ Honey

- Confidence $\equiv P(Honey|Tea) = 100/200 = 0.50$
- Confidence = 50% $\Rightarrow$ drinking tea has little influence on honey usage.
 - The rule seems uninteresting, but ...
- $P(Honey) = 120/1000 = 0.12$ (Tea drinkers are far more likely to have honey!)

Statistical-Based Measures for Interestingness

- Statistical-Based Measures use statistical dependence information.
- **Lift** = $P(Y|X) / P(Y)$
- $\text{Lift}(A,B) = \text{conf}(A \rightarrow B) / \text{support}(B) = \text{support}(A \cup B) / \text{support}(A) \times \text{support}(B)$
- **Interest** = $P(X,Y) / P(X) P(Y)$
- $\text{Interest}(A,B) = \text{support}(A \cup B) / \text{support}(A) \times \text{support}(B)$

$$\text{Interest}(A,B) \begin{cases} = 1 & \text{if } A \text{ and } B \text{ are independent} \\ > 1 & \text{if } A \text{ and } B \text{ are positively correlated} \\ < 1 & \text{if } A \text{ and } B \text{ are negatively correlated} \end{cases}$$

Example: Lift/Interest

	Coffee	$\overline{\text{Coffee}}$	
Tea	15	5	20
$\overline{\text{Tea}}$	75	5	80
	90	10	100

Association Rule: Tea $\Rightarrow$ Coffee

- Confidence = $P(\text{Coffee}|\text{Tea}) = 0.75 = \text{support}(\{\text{Tea}, \text{Coffee}\}) / \text{support}(\{\text{Tea}\})$
- $P(\text{Coffee}) = 0.9$
- Lift = $0.75/0.9 = 0.8333 (< 1, \text{tea and coffee are negatively correlated})$

Leverage

- Leverage measures how much more (or less) A and B occur together than if they were independent.

$$\text{Leverage}(A \Rightarrow B) = P(A \cap B) - P(A) \cdot P(B)$$

Interpretation

- **Leverage = 0**: A and B are **independent** (not interesting rule).
- **Leverage > 0**: A and B occur together **more** than expected (useful rule).
- **Leverage < 0**: A and B occur together **less** than expected (rare or should be avoided).

Conviction

- How strongly the presence of the antecedent (**A**) predicts the presence of the consequent (**B**).
- It is based on the deviation from the independence of A and B.

$$\text{Conviction}(A \Rightarrow B) = \frac{P(\neg B)}{P(\neg B \mid A)} = \frac{1 - P(B)}{1 - \text{Confidence}(A \Rightarrow B)}$$

Interpretation

- **Conviction = 1**: A and B are **independent** (not interesting rule).
- **Conviction > 1**: A strong **positive** association (useful rule).
- **Conviction < 1**: A **negative** association (rule is misleading or unreliable).